CS342 · VISUAL PROGRAMMING

Chapter 2 Graphical Interfaces

Midterm summary

Muharib

What's inside

01 AWT and Swing

02 Containers and components

03 The window JFrame

04 The content pane and JPanel

05 Layout managers

06 Event management

07 Adapters

08 Dialogs and JOptionPane

01 AWT and Swing

Java offers 2 systems for creating graphical interfaces: the oldest is **AWT** (Abstract Window Toolkit, package java.awt); the most recent (since Java 2) is **Swing**, an improvement of AWT (package javax.swing). Swing is a set of classes used to create objects representing graphic components.

	AWT	Swing
Package	**java.awt**	**javax.swing**
Kind	Heavyweight	Lightweight
Relies on	The local window manager; uses more system resources	Independent of the local window manager: draws the component itself
Look	Differs by platform	The same everywhere

- AWT only has heavy components and keeps only what all platforms share (least common denominator).
- All Swing components are lightweight except JApplet, JDialog, JFrame and JWindow.
- Swing component names start with J: JButton versus Button in AWT.

02 Containers and components

A graphical interface in Java is an assembly of **containers** (Container) and **components** (Component).
• A component is a "visible" part of the user interface (buttons, text, drawing areas). It is a subclass of java.awt.Component.
• A container is a space in which you can position several components (windows, applets). Container is itself a subclass of Component.

Container	What it is
Frame / JFrame	A high-level window with a title, a border and resizing
Panel / JPanel	No appearance of its own and not a stand-alone window; added like a button, with its own layout

Group	Examples
Basic components	JButton, JCheckBox, JRadioButton, JToggleButton, JComboBox, JList, JSlider, JTable, JTree, JProgressBar, JSpinner
Menus	JMenuBar, JMenu, JMenuItem, JCheckBoxMenuItem, JRadioButtonMenuItem, JPopupMenu, JToolBar, JToolTip
Text	JTextField, JPasswordField, JTextArea, JTextPane, JEditorPane
Containers	JFrame, JDialog, JOptionPane, JPanel, JScrollPane, JTabbedPane, JSplitPane, JInternalFrame, JDesktopPane, JWindow

Text component	Used for
JTextField	A single line
JPasswordField	A single line with hidden characters
JTextArea	Several lines
JEditorPane	Several lines of different content: text, HTML
JLabel	Fixed text for display only

03 The window JFrame

Method	What it does
setVisible(true) / show()	Shows the window; without it nothing appears
setSize(w, h)	The size; zero by default
setTitle("...")	The text in the title bar
setLocation(x, y)	The window position on the screen
setBounds(x, y, w, h)	Position and size at once
pack()	Resizes the window to the smallest size that shows all its components
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)	Closing the window ends the program
setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE)	Closes the window without ending the program

Warning: by default, closing a JFrame does not terminate the program, but simply makes the window invisible (as if calling setVisible(false)).

So the window can react to events, we write our own class derived from JFrame and set everything up in the constructor:

```
import javax.swing.*;
public class Window extends JFrame {
    public Window() {
        setTitle("My first window");
        setSize(300, 150);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public static void main(String[] args) {
        Window premFen = new Window();
        premFen.setVisible(true);
        premFen.setBounds(250, 250, 300, 200);
    }
}
```

Positioning relative to the screen size needs a Toolkit from java.awt:

```
Toolkit kit = Toolkit.getDefaultToolkit();
Dimension screenSize = kit.getScreenSize();
int H = screenSize.height;
int W = screenSize.width;
Image img = kit.getImage("icon.gif");
this.setIconImage(img);          // change the window icon
```

04 The content pane and JPanel

JFrame has a complex structure (a root, a content, etc.). Components are added to the **content** part with the method **add**. getContentPane() provides the reference to this content of type Container.

```
Container c = getContentPane();
JPanel c = (JPanel) getContentPane();     // or cast it to JPanel
```

The problem: adding several components without a layout shows only the last one, because add puts them all in the same place. The fix: divide the window with a layout.

Absolute positioning: remove the layout and give each component its own position and size.

```
c.setLayout(null);
button1.setBounds(x, y, width, height);   // x from the left, y from the top
```

05 Layout managers

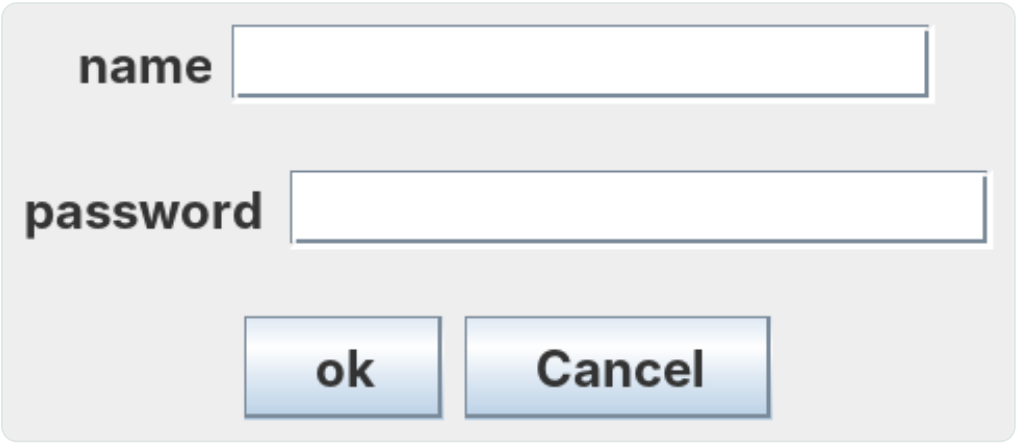
Each container is associated with a layout manager, which controls the location and size of components each time they are displayed. Every container has a default one; change it with setLayout().

Container	Default layout
JFrame	**BorderLayout**
JPanel	**FlowLayout**

Layout	How it arranges	Syntax
FlowLayout	Rows, left to right, then top to bottom, each component at its preferred size; centered by default	**new FlowLayout(FlowLayout.LEFT)**
BorderLayout	5 zones: NORTH, SOUTH, EAST, WEST, CENTER; one component per zone	**add(b, BorderLayout.NORTH)**
GridLayout	A grid of rows and columns; all cells the same size; each component fills its cell	**new GridLayout(3, 2)**

- FlowLayout hides components that do not fit and suits containers with few components. Alignment: LEFT, RIGHT or CENTER.
- BorderLayout: NORTH and SOUTH take their preferred height and the full width; EAST and WEST take their preferred width and the remaining height; CENTER fills the rest. An empty zone has size 0.
- GridLayout: rows or columns can be zero, but not both. The gaps are set with setHgap() and setVgap() (zero by default). Suits a numeric keypad or a toolbar.
- Nesting layouts: the window uses GridLayout or BorderLayout, and each cell holds a JPanel with its own FlowLayout. This builds any window shape the midterm shows.

```
Container p = getContentPane();
p.setLayout(new GridLayout(3, 1));                          // 3 rows
JPanel p1 = new JPanel(new FlowLayout(FlowLayout.CENTER));  // row 1
JPanel p2 = new JPanel(new FlowLayout(FlowLayout.LEFT));    // row 2
JPanel p3 = new JPanel(new FlowLayout(FlowLayout.CENTER));  // row 3
p1.add(l1); p1.add(t1);
p2.add(l2); p2.add(t2);
p3.add(b1); p3.add(b2);
p.add(p1); p.add(p2); p.add(p3);
pack();
```



06 Event management

When the user performs an action (mouse click, item selection, keystroke), an **event** is issued. The component emitting it is the **source**; it transmits the event to one or more **listeners**.
To process an event of type XXXEvent, a listener must implement the interface XXXListener, and must register with the component using addXXXListener(). A component can be its own listener.

Source	Event	Listener	Method
Button click, Enter in a text field	**ActionEvent**	**ActionListener**	**actionPerformed(ActionEvent e)**
Value change in a text box	**TextEvent**	**TextListener**	**textValueChanged(TextEvent e)**
Item selection in a list	**ItemEvent**	**ItemListener**	**itemStateChanged(ItemEvent e)**
Focus gained or lost	**FocusEvent**	**FocusListener**	**focusGained** · **focusLost**
The mouse	**MouseEvent**	**MouseListener**	5 methods: mouseClicked, mouseEntered, mouseExited, mousePressed, mouseReleased

Way 1: the window is the listener. The class implements ActionListener and registers with this. With several buttons, tell them apart with e.getSource().

```
public class Window7 extends JFrame implements ActionListener {
    private JButton b1 = new JButton("OK");
    private JButton b2 = new JButton("To leave");
    private JTextField t = new JTextField();
    public Window7() {
        Container c = this.getContentPane();
        c.setLayout(new BorderLayout());
        c.add(b1, BorderLayout.CENTER);
        c.add(b2, BorderLayout.SOUTH);
        c.add(t, BorderLayout.NORTH);
        b1.addActionListener(this);      // the 2 buttons share one listener
        b2.addActionListener(this);
    }
    public void actionPerformed(ActionEvent e) {
        if ((JButton) e.getSource() == b1) t.setText("OK button pressed");
        else t.setText("EXIT button pressed");
    }
}
```

Way 2: an inner class per button. A class inside the window class that reaches its fields directly. This is the way the midterm asks for (BConvert and BCancel).

```
b1.addActionListener(new OkListener());      // in the constructor
class OkListener implements ActionListener {  // inside the window class
    public void actionPerformed(ActionEvent e) {
        t.setText("OK button pressed");
    }
}
```

Way 3: an anonymous class. The same as an inner class, but without a name, written directly inside add...Listener.

```
addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        String s = "";
        s += ++nb;
        l2.setText(s);
    }
});
```

The method **Object getSource()** is present in all event classes and returns the source object of the event. getX() and getY() on a MouseEvent return the mouse coordinates.

07 Adapters

A class that implements an interface must redefine **all** its methods. Some listener interfaces have many methods, so Java offers **Adapter** classes: inherit from the adapter and redefine only the methods you need. Naming principle: XXXAdapter corresponds to XXXListener.

	implements MouseListener	extends MouseAdapter
Keyword	implements	extends
You write	All 5 methods, even the unused ones (with empty bodies)	Only the method you need

Slide example: a window counting mouse clicks. With MouseListener you write mouseClicked and leave the other four empty; with MouseAdapter you write mouseClicked only.

Close the program from a button with System.exit(0); inside actionPerformed.

08 Dialogs and JOptionPane

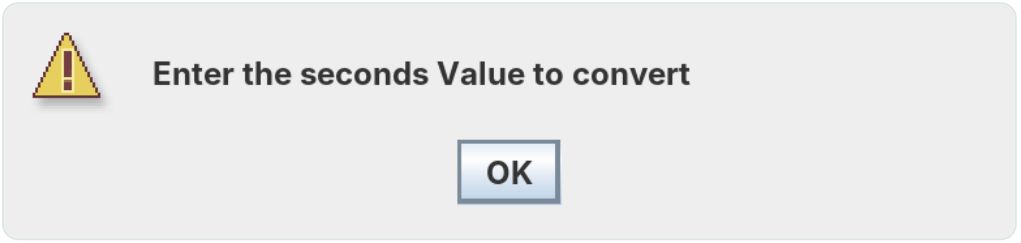
A dialog is a container that you make appear or disappear. Java provides standard dialog boxes through the class **JOptionPane**: message, confirmation, input and option boxes. The class JDialog builds custom dialog boxes.

```
JOptionPane.showMessageDialog(null, "Hello");
```

```
JOptionPane.showMessageDialog(null, "warning", "box title",
JOptionPane.WARNING_MESSAGE);
```

```
//                          parent  message    title        icon type
```

Icon type	Icon
JOptionPane.ERROR_MESSAGE	Error ✕
JOptionPane.INFORMATION_MESSAGE	Information i
JOptionPane.WARNING_MESSAGE	Warning ⚠
JOptionPane.QUESTION_MESSAGE	Question ?
JOptionPane.PLAIN_MESSAGE	No icon



Box	Method	Returns
Message	**showMessageDialog**	Nothing
Input	**showInputDialog(null, "Give a text")**	The String typed, or null if OK was not pressed
Options	**showInputDialog(null, msg, title, type, null, colors, colors[1])**	The Object chosen from the combo box, or null
Confirmation	**showConfirmDialog(null, "Do you want to continue?")**	An int for the button pressed

Confirm buttons	Return value
DEFAULT_OPTION · **YES_NO_OPTION** · **YES_NO_CANCEL_OPTION** · **OK_CANCEL_OPTION**	**YES_OPTION** (0) · **OK_OPTION** (0) · **NO_OPTION** (1) · **CANCEL_OPTION** (2) · **CLOSED_OPTION** (−1)

```
int n = JOptionPane.showConfirmDialog(null, "Do you want to continue?",
                                      "Major incident", JOptionPane.YES_NO_OPTION);
String name = JOptionPane.showInputDialog(null, "Enter your name");
int a = Integer.parseInt(JOptionPane.showInputDialog(null, "First integer"));
```